# TASK 9 – Implement and Test the Billing Program

## 9.1 Complete program:

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000LL

typedef unsigned short meter_t;

static unsigned int consumption(meter_t previous, meter_t
current)
{
   return (unsigned short)(current - previous);
}
```

```
static long long compute_bill(unsigned int units)
{
    long long paise = FIXED_CHARGE_PAISE;

    if (units > SLAB2_UNITS) {
        paise += (SLAB2_UNITS - SLAB1_UNITS) *
RATE2_PAISE;
        paise += (long long)(units - SLAB2_UNITS) *
RATE3_PAISE;
    } else if (units > SLAB1_UNITS) {
        paise += (long long)(units - SLAB1_UNITS) *
RATE2_PAISE;
    }
    return paise;
}

int main(void)
{
    int previous, current;
    printf("Enter previous and current meter readings (0-
65535): ");

    if (scanf("%d %d", &previous, &current) != 2) {
        fprintf(stderr, "Error: enter two integer readings.\n");
        return 1;
```

```
}
    if (previous < 0 || previous > USHRT_MAX ||
        current < 0 || current > USHRT_MAX) {
        fprintf(stderr, "Error: reading out of range.\n");
        return 1;
    }

    unsigned int units = consumption((meter_t)previous,
(meter_t)current);
    long long bill = compute_bill(units);

    printf("Units consumed: %u\n", units);
    printf("Energy charge and fixed charge included.\n");
    printf("Total bill: Rs %lld.%02lld\n", bill / 100, bill %
100);
    return 0;
}
```

This program treats the 16-bit register as a modulo counter. Since two readings alone cannot distinguish rollover from a meter reset or data-entry error, a production system should validate the meter's status and rollover assumptions. The brief explicitly asks for rollover handling, so the code follows that rule.

## 9.2 Test plan and expected results

| Test | Input / condition | Expected result | Status to record |
|---|---|---|---|
| Slab 1 lower boundary | 0 units | ₹50.00 total | Run and fill actual |
| Exactly 100 units | 100 units | ₹50.00 total | Run and fill actual |
| Within slab 2 | 150 units | ₹162.50 total | Run and fill actual |
| Exactly 200 units | 200 units | ₹275.00 total | Run and fill actual |
| Above 200 units | 350 units | ₹950.00 total | Run and fill actual |
| Meter rollover | previous=65400, current=120 | 256 units consumed; bill ₹527.00 | Run and fill actual |
| No consumption | previous=1200, current=1200 | 0 units; bill ₹50.00 | Run and fill actual |
| Invalid letters | previous=a, current=120 | Reject input; no bill | Run and fill actual |
| Negative reading | previous=-1, current=120 | Reject input; no bill | Run and fill actual |
| Out-of-range reading | previous=100, current=70000 | Reject input; no bill | Run and fill actual |

Rollover bill calculation: 256 units means 100 units free, 100 units × ₹2.25 = ₹225, and 56 units × ₹4.50 = ₹252. Energy charge = ₹477; plus ₹50 fixed charge gives ₹527.00. This corrected calculation should be used in the test table.

## 9.3 Five-bill hand calculation

| Household | Units | Energy charge | Fixed charge | Total |
|---|---|---|---|---|
| A | 0 | ₹0.00 | ₹50.00 | ₹50.00 |
| B | 100 | ₹0.00 | ₹50.00 | ₹50.00 |
| C | 150 | ₹112.50 | ₹50.00 | ₹162.50 |
| D | 200 | ₹225.00 | ₹50.00 | ₹275.00 |
| E | 350 | ₹900.00 | ₹50.00 | ₹950.00 |
| Total | — | ₹1,237.50 | ₹250.00 | ₹1,487.50 |

Array-based slab alternative: store slab limits and rates in arrays, then loop over s slabs. Time per bill becomes O(s), and extra storage is O(s) for the tariff table. This is easier to extend when many slabs exist. A fixed sequence of if statements is simpler and effectively constant time when there are only three slabs. A production design should also reject negative unit counts and validate the meter-reading scenario.